

Projeto – fase 2



1. Introdução

Na segunda fase do projeto, o sistema evolui significativamente face à Fase 1, tanto ao nível da estrutura das mensagens trocadas entre cliente e servidor como ao nível da arquitetura de execução.

Nesta fase, deixa-se de utilizar a comunicação em texto livre e passa a existir um **protocolo de comunicação estruturado**, inspirado no modelo **RPC** (Remote Procedure Call). Cada mensagem trocada deverá ter um formato bem definido, distinguindo claramente pedidos (*requests*) de respostas (*responses*), incluindo, entre outras informações, um código de operação. O sistema deverá ainda:

- Implementar a abordagem RPC nas comunicações. A lógica de negócio mantém-se no servidor, mas agora deverá estar devidamente integrada com o novo modelo de comunicação estruturada (RPC).
- Utilizar a serialização para codificar e decodificar as mensagens trocadas entre cliente e servidor.
- Suportar mensagens fragmentadas, garantindo a reconstrução correta dos dados recebidos.
- Permitir múltiplos clientes em simultâneo.
- Implementar multiplexação de I/O, assegurando uma gestão eficiente das ligações concorrentes.
- Introduzir um tratamento de erros de nível intermédio, com respostas estruturadas e códigos de erro adequados.

Na Tabela 1, relembram-se os objetivos do projeto a serem alcançados nas várias fases, com destaque para a **segunda fase**. A Fase 2 também é uma oportunidade para os alunos corrigirem eventuais problemas do Projeto da Fase 1.

Objectivo	Fase 1	Fase 2	Fase 3
Lógica de Negócio	X	X	X
Sockets TCP	X	X	X
Mensagens Texto Livre	X		
Serialização/Desserialização de Dados com pickle		X	
Suporte Mensagens Fragmentadas		X	X
Estruturação RPC de Comunicações		X	X
Multiplexação de I/O		X	X
Tratamento de Erros	Básico	Intermédio	Avançado
Múltiplos Clientes em Simultâneo	Não	Sim	Sim
Gestão de Base de Dados			X
Protocolo HTTP			X
Servidor <i>Flask</i>			X
Comunicação segura			X
Uso de API REST externa (Loja da Amazon)			X

Tabela 1: Requisitos e objetivos a cumprir em cada fase do projeto.

O calendário a seguir volta a apresentar a organização das três fases do projeto, incluindo as datas de disponibilização dos enunciados e as respetivas datas de entrega.

Projeto	Enunciado	Data de Entrega
Fase 1	23 de fevereiro	6 de março
Fase 2	9 de março	27 de março
Fase 3	13 de abril	15 de maio

Tabela 2: Calendário de entregas de projetos e datas de enunciados.

2. Requisitos

2.1. Lógica de negócio

O foco desta fase não é modificar o domínio do problema, mas sim evoluir o modelo de comunicação e execução, separando claramente a lógica de negócio da infraestrutura de transporte (serialização, multiplexação e gestão de clientes concorrentes).

2.2. Sockets TCP

Na primeira fase do projeto, foi requerida a implementação de um modelo simples de comunicação baseado em sockets TCP, no qual cliente e servidor permaneciam em loop, gerando um único pedido e a respetiva resposta de cada vez. Sempre que o cliente introduzia um novo comando, era necessário estabelecer uma nova ligação por meio de um socket dedicado (`conn_sock`), responsável pela troca de mensagens entre cliente e servidor durante esse pedido.

Servidor	Cliente
socket de escuta	loop:
loop:	ler comando do utilizador
aceitar ligação	criar socket
receber pedido	ligar ao servidor
processar	enviar pedido
enviar resposta	receber resposta
fechar ligação	mostrar resposta
	fechar ligação

Tabela 3: Modelo de comunicação da Fase 1.

Apesar de simples, este modelo apresenta algumas limitações. A necessidade de criar um novo *socket* dedicado (`conn_sock`) para cada comando enviado pelo cliente implica um overhead adicional na criação e no encerramento de ligações, o que aumenta o tempo necessário para processar cada pedido. Além disso, este mecanismo torna a comunicação menos eficiente, pois não reutiliza ligações previamente estabelecidas entre cliente e servidor.

Em alternativa, o *select()* é uma chamada de sistema usada em programação de redes para monitorizar vários descritores de ficheiros (como *sockets*) simultaneamente. O servidor mantém uma lista de *file descriptors* ativos (neste caso, o *socket* do servidor, os *sockets* dos clientes e a leitura do *input*). O *select()* é chamado num ciclo infinito para verificar quais têm atividade. Quando um *socket* fica pronto, o programa lê e escreve dados nesse *socket* e, depois, volta a chamar *select()*. Isto implica que, como na primeira fase, o servidor não executa várias operações simultaneamente (não há paralelismo real). Ou seja, há concorrência no nível das ligações, mas não há execução simultânea de código. Isto torna o modelo mais simples e seguro para gerir dados partilhados, embora possa ser menos eficiente se cada operação do lado do servidor (pedido-processa-resposta) demorar muito tempo.

Note-se que, ao contrário da Fase 1, a intenção é que os clientes estabeleçam a ligação ao servidor e que esta permaneça aberta até o programa cliente terminar (reutilização do *conn_sock*). Durante a ligação, o cliente pode enviar vários pedidos. O cliente deve aguardar a resposta do servidor antes de enviar um novo pedido.

Segue pseudo-código do funcionamento básico do *select*. Note que deve adaptar esta lógica ao modelo de RPC.

```
sockets = [server_socket]

loop:
    ready = select(sockets)
    for s in ready:
        if s == server_socket:
            aceitar novo cliente
            adicionar socket à lista
        else:
            receber mensagem
            if cliente terminou:
                remover socket
            else:
                processar pedido
```

Nesta fase, passa a ser possível terminar o servidor de forma controlada utilizando a lógica baseada em *select()*. O servidor também deve incluir a entrada padrão (*stdin*) na lista de descritores monitorizados pelo *select()*. Desta forma, o programa pode reagir a comandos introduzidos diretamente na consola do servidor.

Quando o utilizador introduzir o comando:

- exit
- ou
- quit

o servidor deve:

1. terminar o ciclo principal de execução;
2. fechar todos os *sockets* abertos;
3. terminar o programa de forma controlada.

Este mecanismo permite parar o servidor sem precisar interromper o processo com Ctrl+C.

2.3. Mensagens com prefixo de tamanho

Antes de enviar os dados serializados, o emissor deve enviar um inteiro de 4 bytes (network byte order) indicando o tamanho da mensagem serializada. O receptor deve primeiro ler os 4 bytes do tamanho e, depois, invocar a função *receive_all* para ler exatamente esse número de bytes.

tamanhoX (4 bytes)
mensagem serializada (X bytes)

Tabela 4: Formato da mensagem transmitida na rede.

2.4. Suporte a Mensagens Fragmentadas

Ao contrário do que foi assumido na fase1 do projeto, a função *socket.recv(data_length)* não garante que sejam devolvidos exatamente *data_length* bytes. Pode devolver menos bytes do que o esperado, mesmo que a mensagem completa ainda não tenha sido recebida. Assim, para garantir a receção integral de uma mensagem (ver PL02), é necessário invocar a função

de receção repetidamente até atingir o número total de bytes pretendido. Para este efeito, os alunos devem implementar a função `receive_all`. Esta função deverá:

- Invocar `recv()` múltiplas vezes;
- Acumular os dados recebidos;
- Garantir que apenas retorna quando a mensagem completa tiver sido obtida.

A função `recv()` deverá ser utilizada **apenas** dentro de `receive_all`, sendo esta última função a responsável pela receção efetiva das mensagens no restante programa.

2.5.Mensagens Estruturadas

Uma das mudanças nesta fase consiste na adoção de **mensagens serializadas** como mecanismo de comunicação. Ao invés de mensagens de texto, o cliente e o servidor passam a trocar dados estruturados com recurso à serialização (ver PL02). Um pedido vem na forma de uma lista. A resposta do servidor tem também o formato de lista.

2.6.Remote Procedure Call (RPC)

A chamada a procedimentos remotos (RPC – Remote Procedure Call) é um modelo de estruturação de comunicações que permite que um cliente invoque uma função num servidor como se fosse uma chamada local. No entanto, a execução dessa função ocorre remotamente, noutra máquina. Para o programador, a chamada parece uma função normal, mas existe complexidade escondida, como comunicação em rede, serialização de dados e tratamento de falhas. Por exemplo, num programa normal:

```
categoria = cria_categoria("Fruta")
```

Num sistema distribuído com RPC, o cliente faz algo conceptualmente equivalente, mas:

- A função está no servidor
- Os argumentos são enviados pela rede
- O resultado volta pela rede

Para o cliente, deve parecer apenas uma chamada de função (ver **Figura 1**).

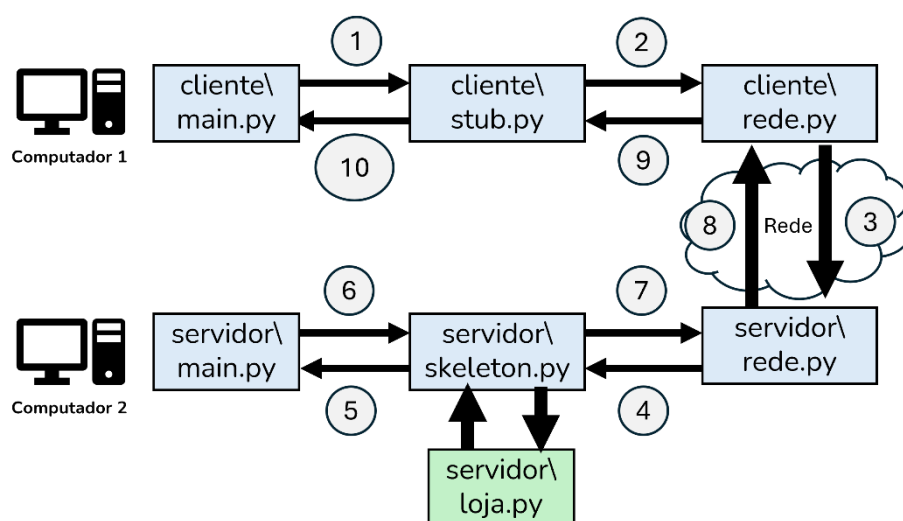


Figura 1: Modelo de estruturação de comunicação Cliente-Servidor RPC.

A figura anterior representa o funcionamento de uma chamada RPC. Na figura são representados os vários módulos (ficheiros .py) envolvidos no cliente e no servidor. Considerando os diferentes módulos e as diversas interações, numeradas na figura, vejamos como a chamada remota é processada:

1. cliente\main.py
O cliente chama uma função remotamente, exemplo: resultado = loja.criar_categoria("Fruta").
2. cliente\stub.py
A função chamada está definida num *stub* – uma função auxiliar que serializa e empacota os dados, e os envia para o servidor.
3. cliente\rede.py → Rede
Os dados serializados são enviados pela rede para o servidor.
4. servidor\rede.py
O servidor recebe os dados pela rede.
5. servidor\skeleton.py
O *skeleton* é o módulo complementar ao *stub*, que executa no lado do servidor. Ele desserializa os dados recebidos e chama a função real no servidor.
6. servidor\main.py
O main.py, do servidor, gere a lógica principal e passa a chamada à parte responsável.
7. servidor\loja.py
Aqui está a função final a executar. Esta função é executada com os argumentos recebidos.
8. Resposta:
O resultado da função (ex: confirmação da compra) é enviado de volta pelo mesmo caminho, agora no sentido inverso.
9. Volta pela rede → cliente\rede.py
10. Resposta é entregue ao *stub* e depois ao cliente\main.py. Por fim, o cliente recebe o resultado como se fosse uma chamada local.

Na implementação da lógica RPC, não é permitido utilizar:

- threading
- multiprocessing
- asyncio
- frameworks RPC existentes

A implementação deve privilegiar **clareza e simplicidade**, mesmo que isso implique alguma repetição de código.

2.7. Autenticação

Nesta fase não é implementado um mecanismo de autenticação real. O perfil e o identificador do utilizador são fornecidos pelo cliente no arranque do programa e assumem-se corretos. O servidor apenas tem a incumbência de verificar se as operações solicitadas estão de acordo com o perfil e o ID. Deixa de ser necessário passar o ID do cliente nos comandos de gestão de carrinho. Deve ser mantido na LISTA_ENCOMENDAS pois um administrador/funcionário tem permissões para ver as encomendas de qualquer cliente. Para clientes registados, o *id_utilizador* corresponde ao *id_cliente* associado ao registo do cliente. Os perfis de administrador e funcionário são considerados já existentes no sistema ou definidos no arranque do servidor. O servidor deve validar o perfil do utilizador e devolver o erro 39920 – OPERACAO_NAO_AUTORIZADA quando a operação não é permitida. O carrinho de compras é associado ao *id_utilizador* que realiza o pedido. Nesta fase do projeto, não é necessário manter uma base de dados de utilizadores nem validar previamente os identificadores fornecidos pelo cliente anónimo, admin e funcionário.

2.8. Tolerância a Falhas e Segurança

Mantêm-se os pressupostos definidos anteriormente:

- O servidor central encontra-se sempre disponível;
- A rede é considerada fiável;

- O ambiente de execução é controlado e confiável.

Não são introduzidos, nesta fase, mecanismos adicionais de segurança ou tolerância a falhas.

3. Protocolo

A comunicação entre cliente e servidor é realizada por meio do envio de mensagens representadas em listas ordenadas. Essas listas são posteriormente serializadas utilizando, obrigatoriamente o **formato pickle**, permitindo a transmissão de objetos Python através da rede. O pedido do cliente é definido da seguinte forma:

```
pedido_do_cliente = [op_code, [lista de argumentos], id_perfil, id_utilizador]
```

```
# pedido cria categoria
pedido = [10100, ["Fruta"], 1, 1]
```

```
# pedido lista categorias
pedido = [10200, [], 1, 1]
```

```
# pedido cria cliente
pedido = [10800, ["username", "email@mail.com", "pass"], 1, 1]
```

Este pedido estruturado corresponde a uma requisição enviada pelo cliente ao servidor para executar uma operação de negócio. O primeiro elemento da lista, *op_code*, representa o código da operação que o servidor deverá executar. Este código identifica de forma inequívoca a ação pretendida, como, por exemplo, listar categorias (LISTA_CATEGORIAS), criar um registo de cliente (CRIAR_CLIENTE) ou atualizar uma informação.

O elemento seguinte é uma lista interior, com **argumento1** até **argumentoN**, que representa os parâmetros necessários para executar a operação. O número e tipo de argumentos varia consoante a operação solicitada.

O penúltimo elemento, perfil, indica o papel ou nível de permissões do cliente no sistema. Este campo é utilizado pelo servidor para verificar se o cliente possui autorização para executar a operação pretendida. O perfil do utilizador é enviado pelo cliente em cada pedido e utilizado pelo servidor para verificar se a operação solicitada é permitida.

Código	Perfil	Descrição
0	CLIENTE_ANONIMO	Utilizador que ainda não se registou ou autenticou no sistema. Tem permissões muito limitadas e apenas pode executar operações públicas. O cliente anónimo pode assumir qualquer ID.
1	CLIENTE_REGISTADO	Utilizador autenticado com conta no sistema. Pode realizar operações associadas a compras e gestão do seu carrinho.
2	FUNCIONARIO	Utilizador interno do sistema com permissões de gestão operacional da loja.
3	ADMINISTRADOR	Utilizador com permissões máximas no sistema.

Tabela 5 – Perfis e códigos associados.

Por fim, o último elemento, *id_utilizador*, identifica unicamente o utilizador que realizou o pedido. O sistema deve manter um registo de utilizadores com um id associado.

Antes de ser enviada pela rede, a lista de pedidos é serializada com o módulo pickle, que converte o objeto Python (neste caso, a lista com todos os seus elementos) num fluxo de bytes. A resposta do servidor é definida da seguinte forma:

```
resposta_do_servidor = [op_code_resposta, [retorno1, ..., retornoN]]
```

```
# resposta cria categorias com sucesso (OK)
resposta = [20100, [<categoria >]]
```

```
# resposta cria categorias sem sucesso (NOK)
```

```
resposta = [30101, [<categoria >]]
```

Esta mensagem é enviada pelo servidor após o processamento do pedido.

- O primeiro elemento, *op_code_resposta*, identifica a operação a que a resposta se refere, permitindo ao cliente associar a resposta ao pedido original.
- O segundo elemento da resposta é uma lista com os dados devolvidos pelo servidor. Estes dados podem ser objetos Python, como listas, dicionários ou outros tipos complexos, e são igualmente serializados com pickle antes de serem enviados.

Os alunos devem continuar a cumprir **todos** os **requisitos de interface** com o cliente estabelecidos na Fase 1 do projeto.

3.1.Códigos de Operação

Cada pedido enviado pelo cliente deve incluir um *op_code* da série 1xxxx. A resposta de sucesso do servidor usa o código correspondente da série 2xxxx. A resposta de erro do servidor utiliza o código correspondente à série 3xxxx. De seguida, apresentam-se os diferentes códigos utilizados pela aplicação.

Área	Operação	Código	Tipo	Descrição
Categorias	Criar categoria	10100	Pedido	Pedido de criação de categoria
Categorias	Criar categoria	20100	Sucesso	Categoria criada com sucesso
Categorias	Criar categoria	30101	Erro	Erro geral ao criar categoria
Categorias	Criar categoria	30110	Erro	Categoria já existe
Categorias	Criar categoria	30111	Erro	Nome da categoria inválido
Categorias	Criar categoria	30112	Erro	Nome da categoria vazio após normalização
Categorias	Listar categorias	10200	Pedido	Pedido de listagem de categorias
Categorias	Listar categorias	20200	Sucesso	Lista de categorias devolvida
Categorias	Listar categorias	30201	Erro	Erro geral ao listar categorias
Categorias	Remover categoria	10300	Pedido	Pedido de remoção de categoria
Categorias	Remover categoria	20300	Sucesso	Categoria removida com sucesso
Categorias	Remover categoria	30301	Erro	Erro geral ao remover categoria
Categorias	Remover categoria	30310	Erro	Categoria não existe
Categorias	Remover categoria	30311	Erro	Categoria contém produtos com stock
Categorias	Remover categoria	30312	Erro	Nome da categoria inválido
Produtos	Criar produto	10400	Pedido	Pedido de criação de produto
Produtos	Criar produto	20400	Sucesso	Produto criado com sucesso
Produtos	Criar produto	30401	Erro	Erro geral ao criar produto
Produtos	Criar produto	30410	Erro	Produto já existe
Produtos	Criar produto	30411	Erro	Categoria não existe
Produtos	Criar produto	30412	Erro	Preço inválido
Produtos	Criar produto	30413	Erro	Quantidade inicial inválida
Produtos	Criar produto	30414	Erro	Nome do produto inválido
Produtos	Listar produtos	10500	Pedido	Pedido de listagem de produtos
Produtos	Listar produtos	20500	Sucesso	Lista de produtos devolvida
Produtos	Listar produtos	30501	Erro	Erro geral ao listar produtos
Produtos	Aumentar stock	10600	Pedido	Pedido de aumento de stock

Produtos	Aumentar stock	20600	Sucesso	Stock aumentado com sucesso
Produtos	Aumentar stock	30601	Erro	Erro geral ao aumentar stock
Produtos	Aumentar stock	30610	Erro	Produto não existe
Produtos	Aumentar stock	30611	Erro	Valor de incremento inválido
Produtos	Atualizar preço	10700	Pedido	Pedido de atualização de preço
Produtos	Atualizar preço	20700	Sucesso	Preço atualizado com sucesso
Produtos	Atualizar preço	30701	Erro	Erro geral ao atualizar preço
Produtos	Atualizar preço	30710	Erro	Produto não existe
Produtos	Atualizar preço	30711	Erro	Novo preço inválido
Clientes	Criar cliente	10800	Pedido	Pedido de criação de cliente
Clientes	Criar cliente	20800	Sucesso	Cliente criado com sucesso
Clientes	Criar cliente	30801	Erro	Erro geral ao criar cliente
Clientes	Criar cliente	30810	Erro	Email já registado
Clientes	Criar cliente	30811	Erro	Nome do cliente inválido
Clientes	Criar cliente	30812	Erro	Email inválido
Clientes	Criar cliente	30813	Erro	Password inválida
Clientes	Listar clientes	10900	Pedido	Pedido de listagem de clientes
Clientes	Listar clientes	20900	Sucesso	Lista de clientes devolvida
Clientes	Listar clientes	30901	Erro	Erro geral ao listar clientes
Carrinho	Adicionar produto	11000	Pedido	Pedido de adicionar produto ao carrinho
Carrinho	Adicionar produto	21000	Sucesso	Produto adicionado ao carrinho
Carrinho	Adicionar produto	31001	Erro	Erro geral ao adicionar produto ao carrinho
Carrinho	Adicionar produto	31010	Erro	Cliente não existe
Carrinho	Adicionar produto	31011	Erro	Produto não existe
Carrinho	Adicionar produto	31012	Erro	Quantidade inválida
Carrinho	Adicionar produto	31013	Erro	Stock insuficiente
Carrinho	Remover produto	11100	Pedido	Pedido de remover produto do carrinho
Carrinho	Remover produto	21100	Sucesso	Produto removido do carrinho
Carrinho	Remover produto	31101	Erro	Erro geral ao remover produto do carrinho
Carrinho	Remover produto	31110	Erro	Cliente não existe
Carrinho	Remover produto	31111	Erro	Produto não existe
Carrinho	Remover produto	31112	Erro	Produto não está no carrinho
Carrinho	Listar carrinho	11200	Pedido	Pedido de listagem do carrinho
Carrinho	Listar carrinho	21200	Sucesso	Carrinho devolvido
Carrinho	Listar carrinho	31201	Erro	Erro geral ao listar carrinho
Carrinho	Listar carrinho	31210	Erro	Cliente não existe
Carrinho	Checkout carrinho	11300	Pedido	Pedido de checkout do carrinho
Carrinho	Checkout carrinho	21300	Sucesso	Checkout realizado com sucesso
Carrinho	Checkout carrinho	31301	Erro	Erro geral no checkout
Carrinho	Checkout carrinho	31310	Erro	Cliente não existe
Carrinho	Checkout carrinho	31311	Erro	Carrinho vazio
Carrinho	Checkout carrinho	31312	Erro	Falha ao criar encomenda
Encomendas	Listar encomendas	11400	Pedido	Pedido de listagem de encomendas

Encomendas	Listar encomendas	21400	Sucesso	Encomendas listadas
Encomendas	Listar encomendas	31401	Erro	Erro geral ao listar encomendas
Encomendas	Listar encomendas	31410	Erro	Cliente não existe

Tabela 6: Códigos de mensagens gerais de negócio.

Código	Erro	Categoria	Quando usar
39900	ERRO_GENERICO	Geral	Erro não classificado; usar apenas como último recurso
39901	OP_CODE_INVALIDO	Protocolo	op_code não existe ou não é suportado
39902	MENSAGEM_MAL_FORMADA	Protocolo	Estrutura da mensagem não segue o formato esperado
39903	PEDIDO_NAO_E_LISTA	Protocolo	Objeto recebido não é uma lista
39904	NUMERO_CAMPOS_INVALIDO	Protocolo	Pedido não tem exatamente 4 campos
39905	ARGUMENTOS_NAO_SAO_LISTA	Protocolo	Segundo elemento do pedido não é lista de argumentos
39906	PERFIL_INVALIDO	Protocolo / autenticação	id_perfil não existe ou está fora do domínio permitido
39907	ID_UTILIZADOR_INVALIDO	Protocolo / autenticação	id_utilizador inválido, inexistente ou mal formatado
39908	SERIALIZACAO_INVALIDA	SerIALIZACAO	Erro ao serializar dados antes de enviar
39909	DESSERIALIZACAO_INVALIDA	SerIALIZACAO	Erro ao desserializar dados recebidos
39910	TAMANHO_MENSAGEM_INVALIDO	Comunicação	Header/tamanho da mensagem inválido, negativo ou inconsistente
39911	MENSAGEM_INCOMPLETA	Comunicação	Não foi possível reconstruir a mensagem completa
39912	LIGACAO_INTERROMPIDA	Comunicação	Cliente/servidor fechou a ligação inesperadamente
39913	TIMEOUT_COMUNICACAO	Comunicação	Leitura ou escrita excedeu o tempo permitido, se implementado
39914	NUMERO_ARGUMENTOS_INVALIDO	Validação	Quantidade de argumentos diferente da esperada
39915	TIPO_ARGUMENTO_INVALIDO	Validação	Tipo de um ou mais argumentos não corresponde ao esperado
39916	VALOR_ARGUMENTO_INVALIDO	Validação	Valor inválido apesar do tipo estar correto
39917	ARGUMENTO_OBRIGATORIO_EM_FALTA	Validação	Faltam dados essenciais para a operação
39918	ARGUMENTO_VAZIO	Validação	String vazia, lista vazia indevida, etc.
39919	ARGUMENTO_FORA_DO_INTERVALO	Validação	Valor numérico fora de limites aceitáveis
39920	OPERACAO_NAO_AUTORIZADA	Autorização	Perfil não tem permissões para executar a operação
39921	UTILIZADOR_NAO_AUTENTICADO	Autorização	Operação exige utilizador autenticado e isso não se verifica
39928	ERRO_INTERNO_SERVIDOR	Execução	Exceção inesperada no servidor

Tabela 7: Códigos de mensagens gerais do sistema distribuído.

Existem códigos específicos para cada operação de negócio e respetivos erros (**Tabela 6**). Para cada operação, o código 3xx01 representa um erro geral da operação. Sempre que a falha corresponder a uma regra de negócio específica do domínio de supermercado/e-commerce, deve ser devolvido um código específico da gama 3xx10–3xx19, em vez do erro geral. As situações “Sem Categorias”, “Sem Produtos”, “Sem Clientes”, “Carrinho Vazio” e “Sem Encomendas”, quando explicitamente definidas no enunciado como respostas válidas, devem ser devolvidas como sucesso e não como erro.

Para além dos códigos específicos de cada operação, o sistema deve suportar códigos genéricos de erro na gama 39xxx (**Tabela 7**). Estes códigos abrangem erros transversais ao protocolo, validação de mensagens, autorização, comunicação, serialização e execução interna. Sempre que possível, o código mais específico aplicável à situação detetada deve ser devolvido. O uso de 39900 deve ser excecional e reservado para erros não classificáveis noutra categoria.

3.2. Especificação em Detalhe

De seguida segue especificação em detalhe do protocolo.

FUNCIONALIDADES DE GESTÃO DE CATEGORIAS		
1 - CRIA_CATEGORIA		
Interação OK (Administrador)	Pedido	[<código:int>, [<nome_categoria:string>], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10100, ["Fruta"], 3, 1]

FUNCIONALIDADES DE GESTÃO DE CATEGORIAS		
	Resposta OK	[<código:int>, [<categoria:Categoria>]]
	Exemplo Resposta OK	[20100, [categoria]]
	Exemplo de Output Cliente OK (Fase 1)	Categoria Fruta criada com sucesso.
2 - LISTA_CATEGORIAS		
Interação OK (Todos os Perfis)	Pedido	[<código:int>, [], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10200, [], 3, 1]
	Resposta OK	[<código:int>, [<lista ordenada de todas as categoria:Categoria>, <lista ordenada de todos os produto:Produto>]]
	Exemplo Resposta OK	[20200, [[categoria1, categoria2], [produto1, produto2, produto3]]]
	Exemplo de Output Cliente OK (Fase 1)	Total Categorias: 4 Total Produtos: 69 1 - Fruta (10 produtos); 2 - Eletrodomésticos (0 produtos); 3 - Livros (58 produtos); 4 - Talho (1 produtos);
3 - REMOVE_CATEGORIA		
Interação OK (Administrador)	Pedido	[<código:int>, [<nome_categoria:string>], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10300, ["Fruta"], 3, 1]
	Resposta OK	[<código:int>, []]
	Exemplo Resposta OK	[20300, []]
	Exemplo de Output Cliente OK (Fase 1)	Categoria Fruta removida com sucesso.

Tabela 8 - Gestão de categorias de produto.

FUNCIONALIDADES DE GESTÃO DE PRODUTOS		
4 - CRIA_PRODUTO		
Interação OK (Administrador, Funcionário)	Pedido	[<código:int>, [<nome_produto: string>, <nome_categoria:string>, <preco:float>, <quantidade:int>], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10400, ["Bananas", "Fruta", 4.50, 400], 2, 1]
	Resposta OK	[<código:int>, [<produto:Produto>]]
	Exemplo Resposta OK	[20400, [produto]]
	Exemplo de Output Cliente OK (Fase 1)	Produto Bananas criado com sucesso.
5 - LISTA_PRODUTOS		
Interação OK (Todos os Perfis)	Pedido	[<código:int>, [], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10500, [], 2, 1]
	Resposta OK	[<código:int>, [<lista_categorias:Categoria>, <lista_produtos:Produto>]]
	Exemplo Resposta OK	[20500, [[categoria1, categoria2], [produto1, produto2, produto3]]]
	Exemplo de Output Cliente OK (Fase 1)	Total Produtos: 3 Total Quantidade: 42 101 - Arroz Integral (Alimentação, 2.49 euros, 15 unidades); 102 - Leite Meio-Gordo (Laticínios, 0.89 euros, 20 unidades); 103 - Detergente Líquido (Limpeza, 4.75 euros, 7 unidades);
6 - AUMENTA_STOCK_PRODUTO		

FUNCIONALIDADES DE GESTÃO DE PRODUTOS		
Interação OK (Administrador, Funcionário)	Pedido	[<código:int>, [<nome_produto:string>, <delta_quantidade:int>], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10600, ["Bananas", 10], 2, 1]
	Resposta OK	[<código:int>, [<produto:Produto>]]
	Exemplo Resposta OK	[20600, [produto]]
	Exemplo de Output Cliente OK (Fase 1)	Stock do produto Bananas aumentado em 10 unidades com sucesso.
7 – ATUALIZA_PRECO_PRODUTO		
Interação OK (Administrador, Funcionário)	Pedido	[<código:int>, [<nome_produto:string>, <novo_preco:float>], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10700, ["Bananas", 10.5], 2, 1]
	Resposta OK	[<código:int>, [<produto:Produto>]]
	Exemplo Resposta OK	[20700, [produto]]
	Exemplo de Output Cliente OK (Fase 1)	O preço do produto Bananas foi atualizado para 10.50 com sucesso.

Tabela 9 - Gestão de produto.

FUNCIONALIDADES DE GESTÃO DE CLIENTES		
8 - CRIA_CLIENTE		
Interação OK (Cliente Anónimo)	Pedido	[<código:int>, [<nome_cliente:string>, <email:string>, <password:string>], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10800, ["Maria Silva", "maria@mail.com", "password"], 0, 999]
	Resposta OK	[<código:int>, [<cliente:Cliente>]]
	Exemplo Resposta OK	[20800, [cliente]]
	Exemplo de Output Cliente OK (Fase 1)	Cliente criado com sucesso com identificador único 23.
9 - LISTA_CLIENTES		
Interação OK (Administrador, Funcionário)	Pedido	[<código:int>, [], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[10900, [], 2, 1]
	Resposta OK	[<código:int>, [<lista_ordenada_de_clientes:Cliente>]]
	Exemplo Resposta OK	[20900, [[cliente1, cliente2]]]
	Exemplo de Output Cliente OK (Fase 1)	Total Clientes: 2 1 - Maria Silva (maria.silva@email.com); 2 - Ana Silva (ana.silva@email.com);

Tabela 10 - Gestão de cliente.

FUNCIONALIDADES DE GESTÃO DE CARRINHO DE COMPRAS		
10 – ADICIONA_PRODUTO_CARRINHO		
Interação OK (Cliente Registado)	Pedido	[<código:int>, [<nome_produto:string>, <quantidade:int>], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[11000, ["Bananas", 10], 1, 23]
	Resposta OK	[<código:int>, [<produto:Produto>]]
	Exemplo Resposta OK	[21000, [produto]]
	Exemplo de Output Cliente OK (Fase 1)	Produto Bananas adicionado com sucesso ao carrinho.
11 – REMOVE_PRODUTO_CARRINHO		
Interação OK (Cliente Registado)	Pedido	[<código:int>, [<nome_produto:string>], <id_perfil:int>, <id_utilizador:int>]

FUNCIONALIDADES DE GESTÃO DE CARRINHO DE COMPRAS		
	Exemplo Pedido	[11100, ["Bananas"], 1, 23]
	Resposta OK	[<código:int>, [<produto:Produto>]]
	Exemplo Resposta OK	[21100, [produto]]
	Exemplo de Output Cliente OK (Fase 1)	Produto Bananas removido com sucesso do carrinho de compras.
12 – LISTA_CARRINHO		
Interação OK (Cliente Registrado)	Pedido	[<código:int>, [], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[11200, [], 1, 23]
	Resposta OK	[<código:int>, [[<lista_categorias:Categoria>], [<lista_produtos:Produto>]]]
	Exemplo Resposta OK	[21200, [[categoria1, categoria2], [produto1, produto2, produto3]]]
	Exemplo de Output Cliente OK (Fase 1)	Total Produtos: 3 Total Quantidade: 8 Total Preço: 154.92 euros 101 - Arroz Integral (1-Alimentação, 2.49 euros, 4 unidades); 205 - Detergente Líquido (3-Limpeza, 4.75 euros, 2 unidades); 310 - Café Torrado (2-Bebidas, 69.99 euros, 2 unidades);
13 – CHECKOUT_CARRINHO		
Interação OK (Cliente Registrado)	Pedido	[<código:int>, [], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[11300, [], 1, 23]
	Resposta OK	[<código:int>, [<encomenda:Encomenda>]]
	Exemplo Resposta OK	[21300, [encomenda]]
	Exemplo de Output Cliente OK (Fase 1)	Checkout de carrinho de compras efetuado com sucesso. Encomenda criada com sucesso a partir do carrinho.

Tabela 11 - Gestão de carrinho de compras de cliente.

FUNCIONALIDADES DE GESTÃO DE ENCOMENDAS		
14 – LISTA_ENCOMENDAS		
Interação OK (Todos os Perfis menos o Cliente Anónimo)	Pedido	[<código:int>, [<id_cliente:int>], <id_perfil:int>, <id_utilizador:int>]
	Exemplo Pedido	[11400, [23], 1, 23]
	Resposta OK	[<código:int>, [[<lista_encomendas:Encomenda>], [<lista_produtos_por_encomenda>]]]
	Exemplo Resposta OK	[21400, [[encomenda1, encomenda2], [[produto1, produto2], [produto1, produto3]]]]
	Exemplo de Output Cliente OK (Fase 1)	Cliente: Marília Silva marilia.silva@mail.com Total Encomendas: 1 Data Encomenda: 2001-01-12 12:12:00 Total Produtos: 3 Total Preço: 159.44 euros Categoria Top: Alimentação, Bebidas ID Encomenda: 1 Total Produtos: 3 Total Quantidade: 8 Total Preço: 159.44 euros 101 - Arroz Integral (Alimentação, 2.49 euros, 4 unidades); 205 - Detergente Líquido (Limpeza, 4.75 euros, 2 unidades); 310 - Café Torrado (Bebidas, 69.99 euros, 2 unidades);

Tabela 12 - Gestão de encomendas.

Todas as respostas do servidor devem ter exatamente dois elementos ao nível superior: o código de resposta e uma lista de retornos. Uma mensagem de resposta do servidor NOK é composta por uma lista de códigos de erro, seguida de uma lista vazia. O cliente é responsável por traduzir os códigos.

4. Estruturação de Dados e Nomes de Atributos

Para permitir a realização de testes automáticos, os estudantes devem manter um conjunto mínimo de atributos nas classes principais do sistema. Os nomes dos atributos indicados abaixo devem ser respeitados exatamente, para garantir a compatibilidade com os testes.

Classe	Atributos obrigatórios	Descrição
Categoria	id_categoria, nome	Identificador único da categoria e respetivo nome
Produto	id_produto, nome, categoria, preco, quantidade	Produto disponível na loja, incluindo categoria associada, preço e stock
Cliente	id_cliente, nome, email, password	Informação básica do cliente registado
Encomenda	id_encomenda, id_cliente, produtos, data, total_preco	Encomenda criada a partir do checkout do carrinho

Notas

- Estes são **apenas os atributos mínimos obrigatórios**.
- Os estudantes podem adicionar outros atributos ou métodos às classes se considerarem necessário.
- Os nomes dos atributos devem ser **exatamente iguais aos indicados**, incluindo maiúsculas e minúsculas.

5. Estruturação de Código

Aplicam-se a mesmas regras da Fase 1 a nível da estruturação de código. Ou seja, pretende-se uma implementação simples, explícita e coerente com os objetivos da disciplina. Assim, não é permitida a utilização de construções avançadas da linguagem que ocultem a lógica fundamental ou introduzam complexidade desnecessária. Especificamente, não é permitida a utilização de:

- **Anotações de tipo (*type hints*)** introduzem uma camada adicional de formalismo na definição de funções e classes.
- **Operador de união de tipos (`|`)**, como em `str | None`, é uma funcionalidade moderna que depende de versões recentes do Python e adiciona complexidade desnecessária num projeto introdutório.
- **@dataclass** automatiza a criação de métodos como o construtor, escondendo detalhes importantes sobre inicialização de objetos. Num projeto inicial, é preferível que o aluno escreva o `__init__` para compreender o processo de criação de instâncias.
- **Decoradores (@algo)**, incluindo `@property`, modificam o comportamento de funções e atributos de forma implícita. O uso de `@staticmethod` é permitido.
- **match/case (pattern matching)** é uma construção moderna que, embora útil, não é essencial para compreender estruturas de decisão básicas e pode criar dependência de versões específicas do Python.
- **Operador walrus (`:=`)** permite atribuições dentro de expressões, tornando o código mais compacto, mas também menos claro para iniciantes.

- **eval()** e **exec()** executam código dinamicamente, o que introduz riscos de segurança e dificulta a análise estática do programa. Além disso, afastam o foco do controlo explícito da lógica.
- **getattr()** e **setattr()** **dinâmicos** permitem metaprogramação, mas tornam o comportamento do programa menos previsível e mais difícil de seguir.

6. Entrega

A entrega da fase 2 do projeto tem de ser feita de acordo com as seguintes regras:

- Colocar todos os ficheiros do projeto, bem como um ficheiro README, num único ficheiro ZIP. O nome do ficheiro será AD-XX-fase2.zip (XX é o nome do grupo).
- Submeter o ficheiro AD-XX-fase2.zip na página da disciplina no Moodle de Ciências@ULisboa, utilizando a atividade disponibilizada para tal. Apenas **um** dos elementos do grupo deve submeter.

Os programas são testados no ambiente Linux dos laboratórios das aulas. Qualquer trabalho entregue que não execute corretamente no ambiente dos laboratórios não será considerado para avaliação. Recomenda-se que usem os laboratórios para o desenvolvimento do projeto e que testem os programas nesse mesmo ambiente.

O prazo de entrega é dia 27/3/2026, até às 23:59 horas.

Após esta data, a submissão do trabalho através do Moodle deixará de ser permitida.

Não são aceites entregas por email.

7. Plágio

Aplicam-se a mesmas regras da Fase 1 a nível de questões de plágio. Ou seja, não é permitido aos alunos partilhar códigos com soluções, ainda que parciais, de qualquer parte do projeto com outros alunos (nem através do Fórum da disciplina, nem por qualquer outro meio). Além disso, todos os códigos serão submetidos a um verificador de plágio. Caso seja encontrada alguma irregularidade, os projetos de todos os alunos envolvidos serão anulados e o caso será reportado aos órgãos responsáveis em Ciências@ULisboa.

Chamamos a atenção para o facto de as plataformas generativas baseadas em Inteligência Artificial (e.g., o ChatGPT e o GitHub Co-Pilot) (1) gerarem um número limitado de soluções diferentes para o mesmo problema, (2) não resolverem corretamente as alíneas descritas no projeto, e (3) incluírem padrões característicos deste tipo de ferramenta, os quais podem vir a ser detetáveis pelos verificadores de plágio. Desta forma, recomendamos fortemente que os alunos não submetam trechos de código gerados por este tipo de ferramenta a fim de evitar riscos desnecessários.

Por fim, é responsabilidade de cada aluno garantir que a sua *home*, as suas diretorias e os seus ficheiros de código estão protegidos contra a leitura de outras pessoas (que não o utilizador dono dos mesmos). Por exemplo, se os ficheiros estiverem gravados na sua área de aluno nos servidores de Ciências@ULisboa, então todos os itens mencionados anteriormente devem ter as permissões de acesso 700. Se os ficheiros estiverem no GitHub, garantam que o conteúdo do vosso repositório não esteja visível publicamente. Caso contrário, a sua participação num eventual plágio será considerada ativa.